

# Large Margin Nearest Neighbor (LMNN) Classification

Based on Section 3 of the provided text

April 25, 2025

# Outline

- 1 Introduction: The LMNN Model
- 2 Intuition and Terminology
- 3 Loss Function
- 4 Local Versus Global Distances
- 5 Convex Optimization
- 6 Energy Based Classification
- 7 Summary

## Building on Distance Metric Learning

- Goal: Learn a Mahalanobis distance metric  
 $\mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) = (\vec{x}_i - \vec{x}_j)^\top \mathbf{M}(\vec{x}_i - \vec{x}_j)$  where  $\mathbf{M} \succeq 0$ .
- $\mathbf{M}$  is learned from labeled data.
- Builds on existing approaches (reviewed in Section 2):
  - **MMC**: Formulate estimation as convex optimization ( $\mathbf{M} \succeq 0$ ).
  - **POLA**: Maximize classification margin.
  - **NCA**: Specifically designed to improve  $k$ NN classification.
- LMNN combines these strengths.

## LMNN: Core Ingredients

The three essential ingredients of our model are:

- 1 **Convex Loss Function:** Facilitates efficient and reliable optimization.
- 2 **Margin Maximization:** Improves robustness to noise.
- 3 **Constraints from  $k$ NN Classification:** Directly targets the goal of improving  $k$ NN accuracy.

We achieve this by learning a linear transformation  $\mathbf{L}$  (and thus  $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$ ) of the input space.

## Intuition for Robust $k$ NN

Two simple intuitions guide the model:

- 1 **Neighborhood Homogeneity:** Each training input  $\vec{x}_i$  should share the same label  $y_i$  as its  $k$  nearest neighbors (in the learned metric).
- 2 **Class Separation:** Training inputs with different labels should be widely separated.

These are balanced by competing terms in the loss function:

- Term 1: Penalizes large distances between nearby inputs with the same label. (Pull)
- Term 2: Penalizes small distances between inputs with different labels. (Push)

To make this precise, we need specific terminology.

## Key Terminology

- **Target Neighbors** ( $j \rightsquigarrow i$ ):
  - Training inputs  $\vec{x}_j$  that we \*desire\* to be closest to  $\vec{x}_i$ .
  - Fixed *a priori* (e.g.,  $k$  nearest neighbors of  $\vec{x}_i$  with the same class label  $y_i$  using Euclidean distance).
  - Simplifies learning by specifying desired local structure.
  - Relation  $j \rightsquigarrow i$  is not symmetric.
- **Importors**:
  - Training inputs  $\vec{x}_l$  with a different label ( $y_l \neq y_i$ ) that "invade" the perimeter around  $\vec{x}_i$  established by its target neighbors.
  - Goal of learning: minimize the number of importors.
- **Margin**:
  - Maintain a large distance between importors and the perimeters defined by target neighbors.
  - Improves robustness to noise.
  - Gives rise to the name: **Large Margin Nearest Neighbor**

## Defining Importors and Margin

An importor  $\vec{x}_l$  (with  $y_l \neq y_i$ ) relative to  $\vec{x}_i$  and its target neighbor  $\vec{x}_j$  is defined by the inequality:

$$\|\mathbf{L}(\vec{x}_i - \vec{x}_l)\|^2 \leq \|\mathbf{L}(\vec{x}_i - \vec{x}_j)\|^2 + 1$$

Equivalently, using the Mahalanobis distance

$$\mathcal{D}_{\mathbf{M}}(\vec{u}, \vec{v}) = \|\mathbf{L}(\vec{u} - \vec{v})\|^2:$$

$$\mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_l) \leq \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) + 1 \quad (\text{Eq. 10})$$

This means  $\vec{x}_l$  is closer to  $\vec{x}_i$  than  $\vec{x}_j$  by a margin of 1 (in squared distance).

Figure: \*

Figure 1: Schematic Illustration, Before training (left), target

## The LMNN Loss Function

The loss function consists of two competing terms:

- $e_{\text{pull}}(\mathbf{L})$ : Penalizes large distances between  $\vec{x}_i$  and its target neighbors  $\vec{x}_j$ . (Attraction)
- $e_{\text{push}}(\mathbf{L})$ : Penalizes small distances between  $\vec{x}_i$  and differently labeled examples  $\vec{x}_l$ . (Repulsion)

Using  $\mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) = \|\mathbf{L}(\vec{x}_i - \vec{x}_j)\|^2$ :

- **Pull Term (Eq. 11):**

$$e_{\text{pull}}(\mathbf{L}) = \sum_i \sum_{j \rightsquigarrow i} \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j)$$

*Important:* Only penalizes distances to *target* neighbors, not all similarly labeled inputs. Focuses on local structure.

- **Push Term (Eq. 12):** Uses hinge loss  $[z]_+ = \max(z, 0)$  to penalize violations of Eq. 10.



## Combined Loss and Analogy to SVMs

The combined loss function balances pulling and pushing:

$$e(\mathbf{L}) = (1 - \mu)e_{\text{pull}}(\mathbf{L}) + \mu e_{\text{push}}(\mathbf{L}) \quad (\text{Eq. 13})$$

- $\mu \in [0, 1]$  is a weighting parameter (often set to 0.5).
- The unit margin is a convention (scales  $\mathbf{L}$ ).

### **Analogy to SVMs:**

- Both have competing terms: a penalty on the model parameters (norm of weights /  $\mathbf{L}$ ) and a hinge loss on margin violations.
- Hinge loss is triggered only by problematic examples (near boundary in SVM, importors in LMNN).
- Can be formulated to depend on inputs only via inner products (allowing Kernel Trick - Section 5.3)

## Local vs. Global Distances

- LMNN's focus on pulling only *target neighbors* (local) is a key distinction.
- Many previous methods (LDA, RCA, MCC) try to shrink distances between *all* examples in the same class (global).
- This global focus can be problematic for classes with multimodal support or complex, non-globally-compact structures.

Figure: Toy Data Example

## Optimization: From $\mathbf{L}$ to $\mathbf{M}$

- The loss function  $e(\mathbf{L})$  is *not* convex in the elements of  $\mathbf{L}$ .
- Gradient descent in  $\mathbf{L}$  can get stuck in local minima.
- Solution: Reformulate in terms of  $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$ .
- The Mahalanobis distance  $\mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j)$  is linear in  $\mathbf{M}$ .
- Substituting into the loss (Eq. 13):

$$e(\mathbf{M}) = (1-\mu) \sum_{i,j \rightsquigarrow i} \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) + \mu \sum_{i,j \rightsquigarrow i} \sum_{l, y_l \neq y_i} [1 + \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_j) - \mathcal{D}_{\mathbf{M}}(\vec{x}_i, \vec{x}_l)]$$

- This  $e(\mathbf{M})$  is a piecewise linear, **convex** function of  $\mathbf{M}$ .
- We must add the constraint  $\mathbf{M} \succeq 0$  (positive semidefinite) to ensure it's a valid Mahalanobis metric.

## Optimization: Semidefinite Program (SDP)

The convex optimization can be formulated as an SDP by introducing slack variables  $\xi_{ijl} \geq 0$  for each triplet  $(i, j \rightsquigarrow i, l \text{ with } y_l \neq y_i)$ . The slack  $\xi_{ijl}$  measures the violation of the margin constraint.

Minimize:

$$\begin{aligned}
 (1 - \mu) \sum_{i, j \rightsquigarrow i} (\vec{x}_i - \vec{x}_j)^\top \mathbf{M} (\vec{x}_i - \vec{x}_j) \\
 + \mu \sum_{i, j \rightsquigarrow i} \sum_{l, y_l \neq y_i} \xi_{ijl}
 \end{aligned}$$

Subject to:

$$(1) \quad (\vec{x}_i - \vec{x}_j)^\top \mathbf{M} (\vec{x}_i - \vec{x}_j) - (\vec{x}_i - \vec{x}_l)^\top \mathbf{M} (\vec{x}_i - \vec{x}_l) \geq 1 - \xi_{ijl}$$

for all  $i, j \rightsquigarrow i, l \text{ with } y_l \neq y_i$

## Solving the SDP

- General-purpose SDP solvers can be slow, scaling poorly with the number of constraints.
- The number of constraints in LMNN can be large (triplets  $(i, j, l)$ ).
- However, most slack variables  $\xi_{ijl}$  are zero at the optimum (most points are far from the margin boundary).
- This sparsity can be exploited.
- Our implementation uses a specialized solver:
  - Combination of sub-gradient descent in  $\mathbf{L}$  and  $\mathbf{M}$ .
  - Projection onto the positive semidefinite cone after  $\mathbf{M}$  updates.
  - Starts with a smaller set of active constraints and expands.
  - Empirically much faster than generic solvers.
- Code available:

<https://github.com/kimi-moonshot/moonshot-ai>

## Energy-Based Classification

- The learned matrix  $\mathbf{M}$  can be used directly for  $k$ NN classification.
- Alternatively, use the LMNN loss function itself as an "energy" for classification (inspired by energy-based models).
- For a test example  $\vec{x}_t$ :
  - Consider each possible hypothetical label  $y_t$ .
  - For a given  $y_t$ , identify target neighbors of  $\vec{x}_t$  among the training data (e.g.,  $k$  training examples with label  $y_t$  closest to  $\vec{x}_t$ ).
  - Compute an "energy" score for  $\vec{x}_t$  under this hypothetical label, incorporating interactions with *\*all\** training data using the learned  $\mathbf{M}$ .

## Energy-Based Classification (cont.)

The energy for a test example  $\vec{x}_t$  with hypothetical label  $y_t$  (using learned  $\mathbf{M}$ ) involves three terms (Eq. 16):

- 1 **Pull Term:** Distance from  $\vec{x}_t$  to its  $k$  target neighbors (given  $y_t$ ).

$$(1 - \mu) \sum_{j \rightsquigarrow t} \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_j)$$

- 2 **Push Term (Test  $\rightarrow$  Train):** Hinge loss for training importors invading  $\vec{x}_t$ 's perimeter.

$$\mu \sum_{j \rightsquigarrow t} \sum_{l, y_l \neq y_t} [1 + \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_j) - \mathcal{D}_{\mathbf{M}}(\vec{x}_t, \vec{x}_l)]_+$$

- 3 **Push Term (Train  $\rightarrow$  Test):** Hinge loss for  $\vec{x}_t$  invading other training examples' perimeters.

## Summary of LMNN

Large Margin Nearest Neighbor (LMNN) is a distance metric learning approach for  $k$ NN classification:

- Learns a Mahalanobis metric  $\mathbf{M} = \mathbf{L}^\top \mathbf{L}$ .
- Based on optimizing two goals:
  - Pulling target neighbors closer.
  - Pushing differently-labeled examples (importors) away with a margin.
- Formulated as a convex optimization problem over positive semidefinite matrices ( $\mathbf{M} \succeq 0$ ).
- The optimization is expressed as a Semidefinite Program (SDP).
- Designed to handle locally structured data by focusing on target neighbors.
- Can be used for standard  $k$ NN with the learned  $\mathbf{M}$  or via an



## References

- Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
- Chopra, S., Ranzato, M., & LeCun, Y. (2005). Learning a Similarity Metric Discriminatively, with Application to Face Verification. *CVPR*.
- Goldberger, J., Hinton, G., Roweis, S., & Salakhutdinov, R. (2005). Neighbourhood Components Analysis. *NIPS*.
- Schölkopf, B., & Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Vandenberghe, L., & Boyd, S. (1996). Semidefinite Programming. *SIAM Review*, 38(1), 49-95.

Introduction: The LMNN Model  
Intuition and Terminology  
Loss Function  
Local Versus Global Distances  
Convex Optimization  
Energy Based Classification  
Summary  
References

# Thank You

Questions?